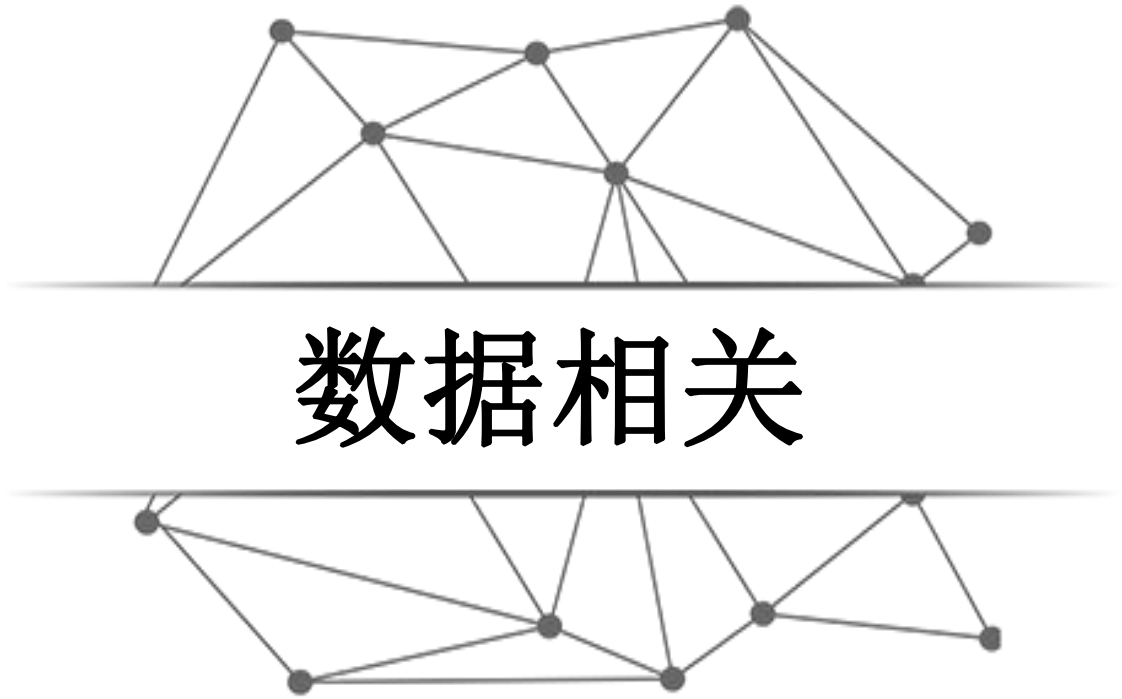
# 线性回归与逻辑回归




致谢：梁威鹏


线性回归与逻辑回归
以西瓜数据为例解释概念
线性回归
逻辑回归
实验步骤
梯度下降



# 数据相关

如图所示，假定我们收集了一批西瓜的数据。




【示例、样本】：对一个西瓜的描述

eg：x1=（色泽=青绿；根蒂=蜷缩；敲声=浊响）

【数据集】：一组记录的集合，一组“对西瓜的描述”的集合。

eg：D={x1,x2,x3...xm}

【属性、特征】：

eg：色泽; 根蒂; 敲声

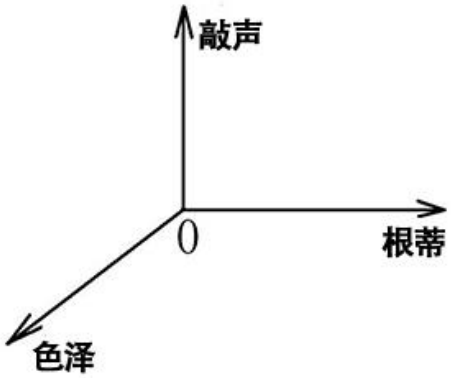
【属性值】：

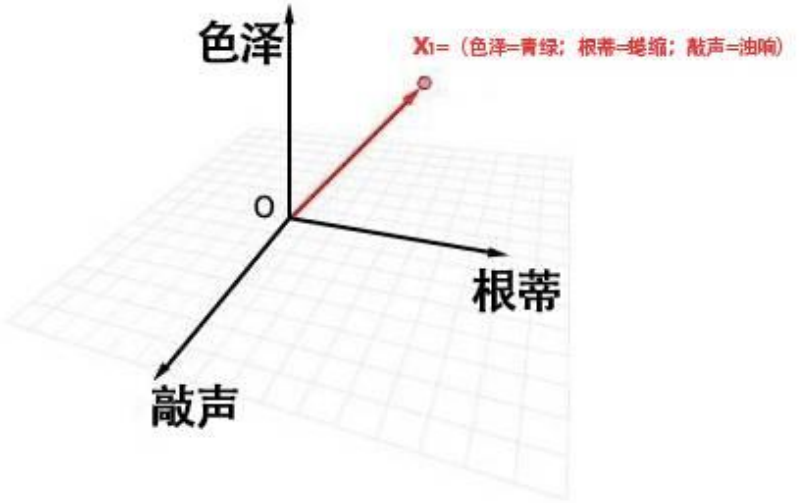
eg：　=青绿;　=蜷缩; =浊响

如图所示，假定我们收集了一批西瓜的数据。




【属性空间、样本空间、输入空间】：以西瓜的三种属性为三个坐标轴，建立坐标系得到的空间。（此图系在假设属性值为连续而非离散的基础上绘制）




【特征向量】：西瓜的三个属性的属性值可以在属性空间坐标轴上找到属于自己的坐标，由此找到空间中符合三个属性值的一个坐标点。由于空间中每一个点对应一个坐标向量，故一个示例可以在d个属性围成的d维属性空间中表示成一个向量。

【预测】：依靠机器学习得到的模型，对新示例进行结果判断。

【标记空间、输出空间】：所有标记的集合。

- 当标记值是离散值的时候，Y={y1，y2，y3，...yj，...yn}（此时y的序号j并不代表第几个样例，而代表标记的第几个取值，n表示标记可以取n个值）在好瓜与坏瓜的问题上，n=2，即标记可以取两个值。
  - eg：Y={好瓜，坏瓜}
- 当标记值是连续值的时候，Y={yi=f(xi)}（此时yi则代表在取序号为i的示例xi时，标记的取值示例xi做自变量的函数f(xi)），如对西瓜的色泽、根蒂、敲声进行量化的统计，则可归纳出某种函数f(xi)，用以表达西瓜的成熟程度值，从而判断西瓜的好坏。

【分类】：预测的是离散值。

- eg：判断西瓜的好坏。

【回归】：预测的是连续值。

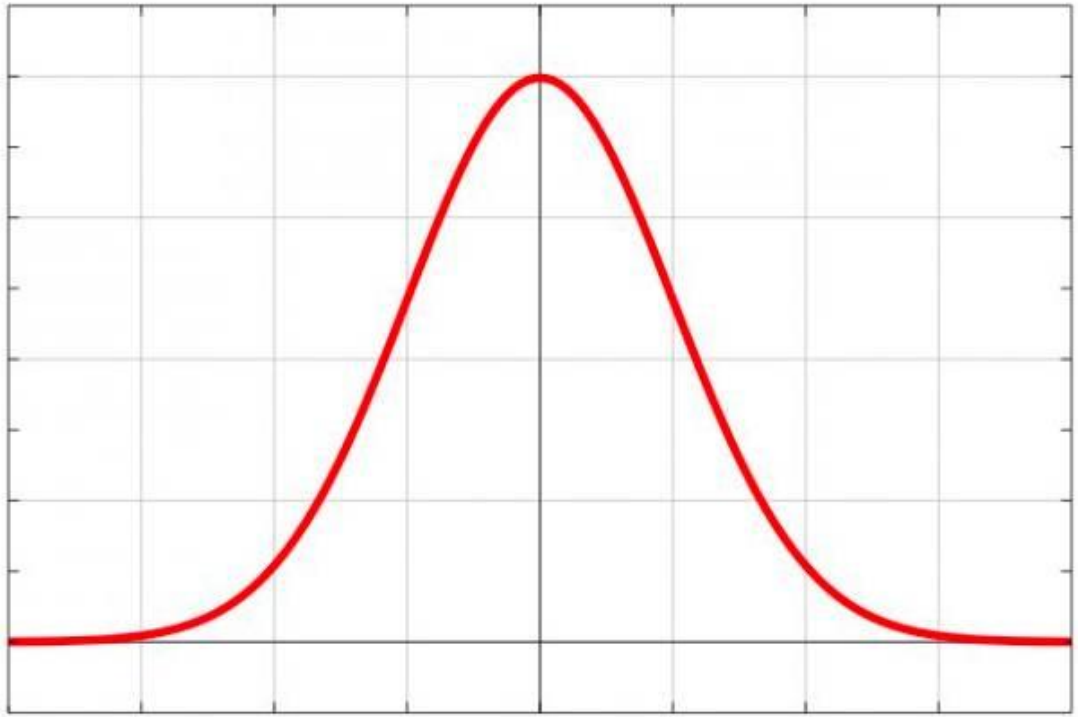
- eg：判断西瓜的成熟度。

## 机器学习的目标

机器学习的目标是使学得的模型可以更好的适用于“新样本”。

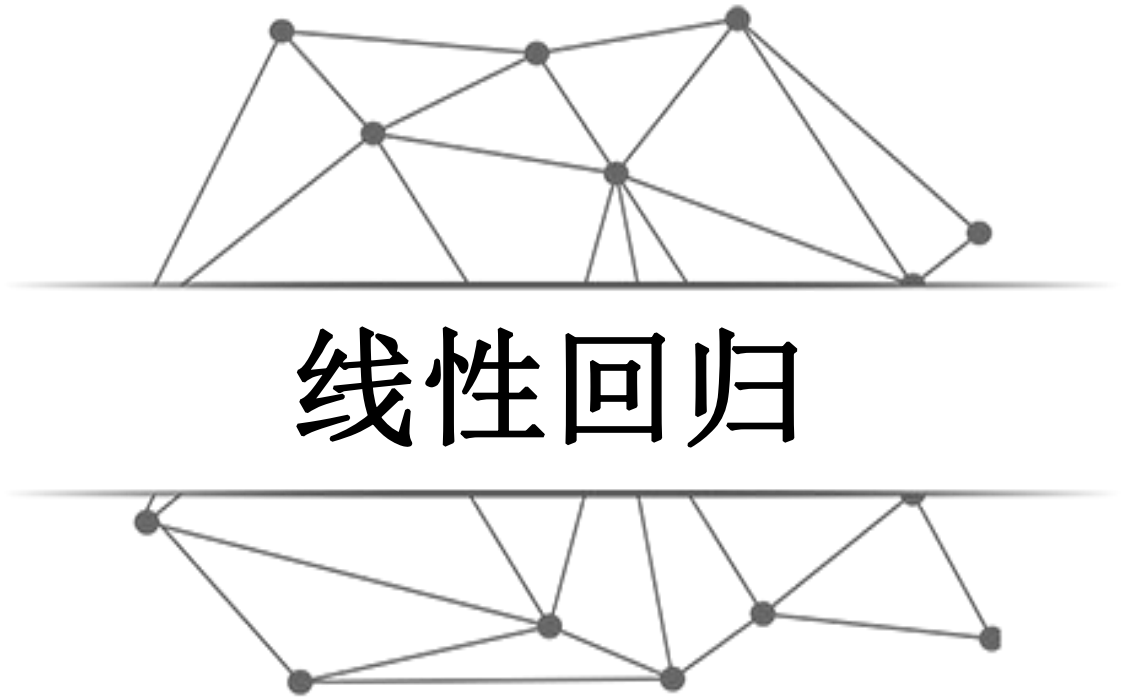
【泛化】：学得模型适用于新样本的能力。

泛化能力一定程度上体现出假设与真相之间的差距。



- 假设样本空间的全体样本服从一个未知分布D。

- 一般而言，训练样本越多，得到关于D的信息就越多，越有可能通过学习获得强泛化能力的模型。



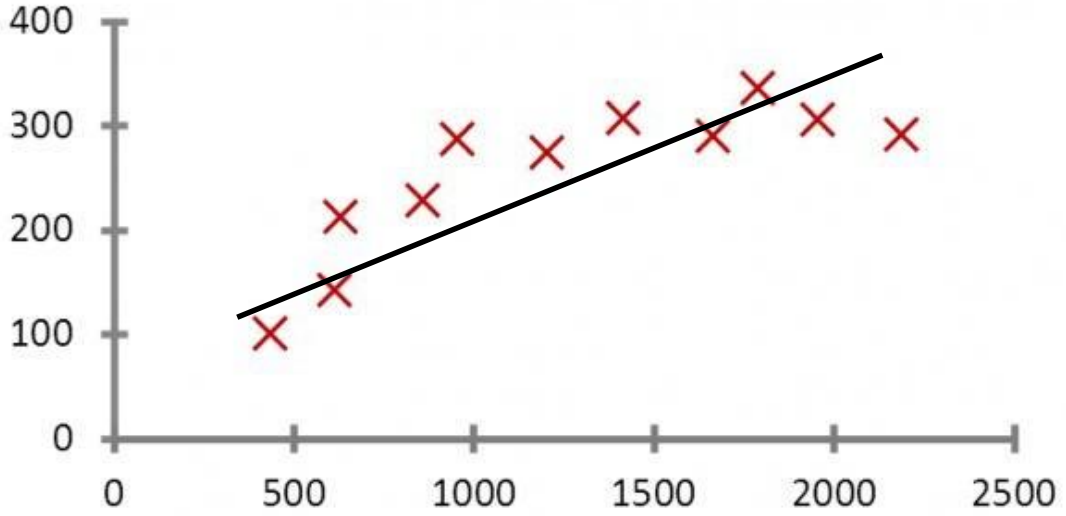
# 线性回归

# 线性回归

- 线性回归(Linear Regression)是利用数理统计中回归分析，来确定两种或两种以上变量间相互依赖的定量关系的一种统计分析方法。
- 线性回归利用称为线性回归方程的最小平方函数对一个或多个自变量和因变量之间关系进行建模。这种函数是一个或多个称为回归系数的模型参数的线性组合。只有一个自变量的情况称为简单回归,大于一个自变量情况的叫做多元回归。

# 线性回归

线性回归：使用形如y=wTx+b的线性模型拟合数据输入和输出之间的映射关系的。



# 线性回归的实际用途

线性回归有很多实际的用途，分为以下两类：

1. 如果目标是预测或者映射，线性回归可以用来对观测数据集的y和X 的值拟合出一个预测模型。当完成这样一个模型以后，对于一个新增的X值， 在没有给定与它相配对的y的情况下，可以用这个拟合过的模型预测出一个y值。

2. 给定一个变量y和一些变量$x_1,x_2...x_n$，这些变量有可能与y相关，线性回归分析可以用来量化y与 $X_i$ 之间相关性的强度，评估出与y不相关的 $X_i$，并识别出哪些 $X_i$ 的子集包含了关于y的冗余信息。

# 线性回归的应用

- 背景：与西瓜成熟度密切相关的除了西瓜的密度，还有西瓜的含糖量。我们可以根据已知的西瓜的密度和西瓜的含糖量进行线性回归，继而可以对在已知西瓜的密度、西瓜的含糖量的前提下，对西瓜的成熟度进行预测。
- 应用目标：对西瓜的两个特征与西瓜的成熟度建立回归方程，并依据回归方程对成熟度进行预测。并解释密度跟含糖量哪个指标更重要?
- 数学建模：

  假设函数: $h_\theta(x) = \theta_0 x_0 + \theta_1 x_1 + \ldots + \theta_n x_n$

  损失函数: $J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2$

  目标: $\min J(\theta_0, \theta_1, \ldots, \theta_n)$
- 技术路线：sklearn.linear_model.LinearRegression

# Sklearn中的线性回归

技术路线：sklearn.linear_model.LinearRegression 官方文档 中文文档

## sklearn.linear_model.LinearRegression

*class* sklearn.linear_model.LinearRegression(*, *fit_intercept=True*, *normalize='deprecated'*, *copy_X=True*, *n_jobs=None*, *positive=False*) [source]

Ordinary least squares Linear Regression.

LinearRegression fits a linear model with coefficients w = (w1, ..., wp) to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation.

疑问：最小二乘法?

### 1.1.1. 普通最小二乘法

`LinearRegression` 拟合一个带有系数 $w = (w_1, \ldots, w_p)$ 的线性模型，使得数据集实际观测数据和预测数据（估计值）之间的残差平方和最小。其数学表达式为:

$$\min_w ||Xw - y||_2^2$$

`LinearRegression` 会调用 `fit` 方法来拟合数组 X， y，并且将线性模型的系数 $w$ 存储在其成员变量 `coef_` 中:

```
>>> from sklearn import linear_model
>>> reg = linear_model.LinearRegression()
>>> reg.fit ([[0, 0], [1, 1], [2, 2]], [0, 1, 2])
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=1, normalize=False)
>>> reg.coef_
array([ 0.5,  0.5])
```

# 最小二乘法

最小二乘法：背后的假设和原理（前篇）

$$J(\theta)=\frac{1}{2}\sum_{i=1}^{m}\left(y^{(t)}-\theta^{T}x^{(t)}\right)^{2}$$

最小二乘法原理（中）：似然函数求权重参数

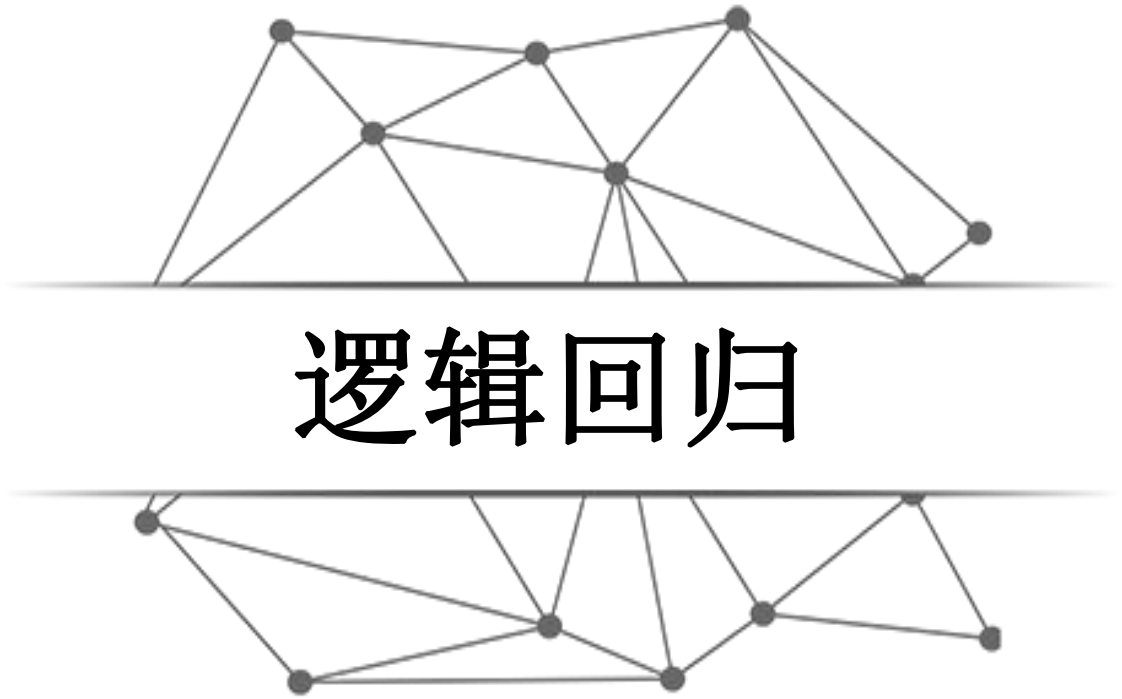
$$\theta=\left(X^{T}X\right)^{-1}X^{T}y$$

最小二乘法原理（后）：梯度下降求权重参数

$$\frac{\partial J(\theta)}{\partial\theta}=-\frac{1}{m}\sum_{i=1}^{m}\left(y^{(i)}-\theta^{T}x^{(i)}\right)x_{j}^{(i)}$$

$$\theta_{j}'=\theta_{j}-\left(-\frac{1}{m}\sum_{i=1}^{m}\left(y^{(i)}-\theta^{T}x^{(i)}\right)x_{j}^{(i)}\right)$$

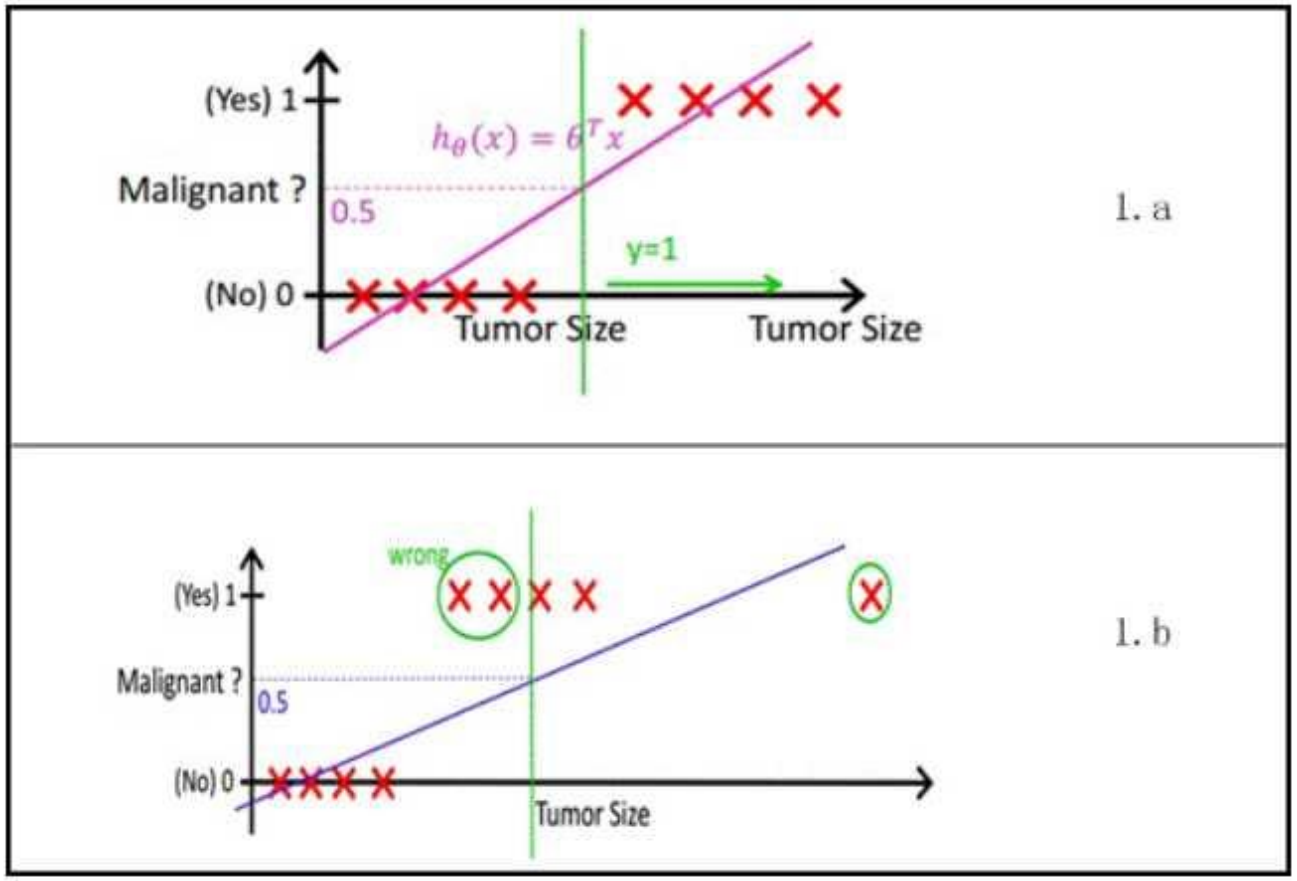
| 梯度下降 | 正规方程 |
| --- | --- |
| 需要选择学习率 α | 不需要 |
| 需要多次迭代 | 一次运算得出 |
| 当特征数量 n 大时也能较好适用 | 需要计算$(X^{T}X)^{-1}$<br>如果特征数量 n 较大则运算代价大，因为矩阵逆的计算时间复杂度为 $O(n^{3})$，通常来说当 n 小于 10000 时还是可以接受的 |
| 适用于各种类型的模型 | 只适用于线性模型，不适合逻辑回归模型等其他模型 |



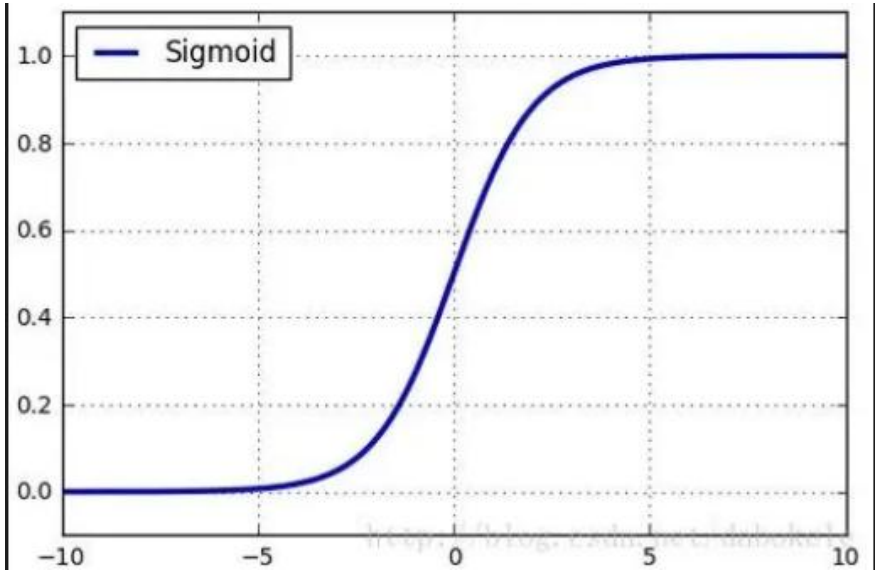
# 逻辑回归

# 逻辑回归——分类问题

既然能够用线性回归预测出连续值结果，那根据结果设定一个阈值是不是就可以解决分类问题了呢?




- 线性回归的结果输出是一个连续值，而值的范围是无法限定的。
- 如果输出结果是 (0,1) 的一个概率值——sigmoid函数



# 逻辑回归——分类问题

- 线性回归的预测函数为$Y=W^TX$，那么逻辑回归的输出$Y= g(W^TX)$，其中y=g(z)函数正是上述sigmoid函数（又称Logistic函数）。

$$g(z) = \frac{1}{1 + e^{-z}}$$

# 逻辑回归 VS 线性回归

- 逻辑回归和线性回归本质上都是得到一条直线。
- 线性回归的直线是尽可能去拟合输入变量X的分布，使得训练集中所有样本点到直线的距离最短；
- 而逻辑回归的直线是尽可能去拟合决策边界，使得训练集样本中的样本点尽可能分离开。因此，两者的目的是不同的。

## 1.1.11. logistic 回归

logistic 回归，虽然名字里有 “回归” 二字，但实际上是解决分类问题的一类线性模型。在某些文献中，logistic 回归又被称作 logit 回归，maximum-entropy classification（MaxEnt，最大熵分类），或 log-linear classifier（对数线性分类器）。该模型利用函数 logistic function 将单次试验（single trial）的可能结果输出为概率。

scikit-learn 中 logistic 回归在 `LogisticRegression` 类中实现了二分类（binary）、一对多分类（one-vs-rest）及多项式 logistic 回归，并带有可选的 L1 和 L2 正则化。

**Examples** 官方文档

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.linear_model import LogisticRegression
>>> X, y = load_iris(return_X_y=True)
>>> clf = LogisticRegression(random_state=0).fit(X, y)
>>> clf.predict(X[:2, :])
array([0, 0])
>>> clf.predict_proba(X[:2, :])
array([[9.8...e-01, 1.8...e-02, 1.4...e-08],
       [9.7...e-01, 2.8...e-02, ...e-08]])
>>> clf.score(X, y)
0.97...
```



# 预测西瓜成熟度

——并解释密度跟含糖量哪个指标更重要?

# 实验步骤

1. 导入依赖包

2. 导入数据集

3. 划分训练集和验证集

4. 使用线性回归对训练集进行训练

5. 预测结果与验证集的label进行对比，
   计算其评价指标

1. 导入依赖包

```
import sklearn
import matplotlib.pyplot as plt
from sklearn import model_selection
from sklearn import linear_model # 导入线性回归模型
from sklearn import metrics # 导入模型度量
import xlrd  # pip install xlrd==1.2.0
import  pandas  as pd
```

```
excel_file = '西瓜数据集.xlsx'
# 读取excel文件里的数据
data = pd.read_excel(excel_file)
```

2. 导入数据集

```
print(data.values)
```

```
[[ 1.     0.697  0.46   0.89 ]
 [ 2.     0.774  0.376  0.9  ]
 [ 3.     0.634  0.264  0.76 ]
 [ 4.     0.608  0.318  0.78 ]
 [ 5.     0.556  0.215  0.58 ]
 [ 6.     0.403  0.237  0.61 ]
 [ 7.     0.481  0.149  0.6  ]
 [ 8.     0.666  0.091  0.48 ]
 [ 9.     0.243  0.268  0.42 ]
 [10.     0.245  0.057  0.33 ]
 [11.     0.343  0.099  0.37 ]
 [12.     0.639  0.161  0.45 ]
 [13.     0.657  0.198  0.4  ]
 [14.     0.36   0.37   0.22 ]]
```

# 实验步骤

1. 导入依赖包

2. 导入数据集

3. 划分训练集和验证集

4. 使用线性回归对训练集进行训练

5. 预测结果与验证集的label进行对比，
   计算其评价指标

3. 划分训练集和验证集

In [46]:
```
x_train, x_test, y_train, y_test = model_selection.train_test_split(x, y, test_size = 0.1)
```

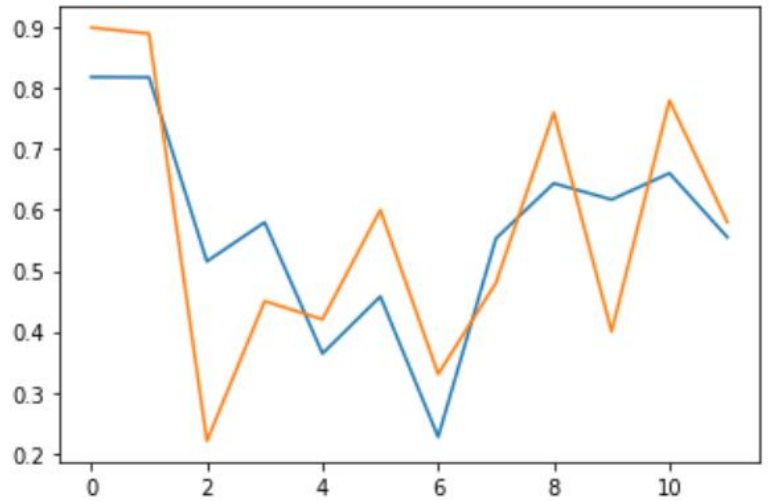
In [50]:
```
# 划分训练集和测试集
print(x_train.shape)
print(y_train.shape)
print(x_test.shape)
print(y_test.shape)
```

```
(12, 2)
(12, 1)
(2, 2)
(2, 1)
```

4. 训练

In [56]:
```
# 训练
L_R=linear_model.LinearRegression()
L_R.fit(x_train,y_train)

# 可视化训练过程
y_train_predict=L_R.predict(x_train)
plt.plot([i for i in range(len(y_train_predict))],y_train_predict)
plt.plot([i for i in range(len(y_train))],y_train)
plt.show()
```



# 实验步骤

1. 导入依赖包

2. 导入数据集

3. 划分训练集和验证集

4. 使用线性回归对训练集进行训练

5. 对验证集进行预测

6. 预测结果与验证集的label进行对比，计算其评价指标

7. 密度跟含糖量哪个指标更重要?

5. 预测

```
In [58]: y_predict=L_R.predict(x_test)
         print(y_predict)
         print(y_test)
```

```
[[0.45915164]
 [0.32514655]]
[[0.61]
 [0.37]]
```

6. 评估

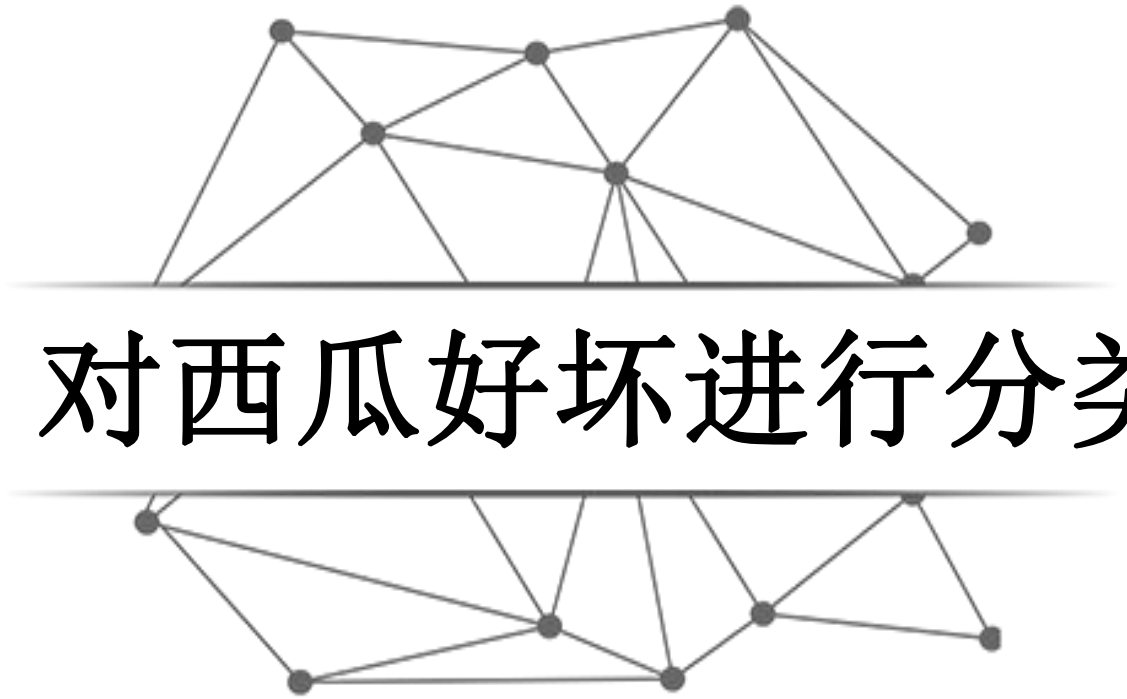
```
SQ_ERR=metrics.mean_squared_error(y_predict,y_test)
print("测试集的平方误差为{}".format(SQ_ERR))
```

```
测试集的平方误差为0.012383529492641 45
```

7. 答案：?

```
In [60]: print(L_R.coef_)
```

```
[[0.72253901 0.65690402]]
```



# 对西瓜好坏进行分类

# 实验步骤

1. 导入依赖包

2. 导入数据集

3. 划分训练集和验证集

4. 使用逻辑回归对训练集进行训练

5. 对验证集进行预测

In [13]:
```
import sklearn
import matplotlib.pyplot as plt
from sklearn import model_selection
from sklearn.linear_model import LogisticRegression
from sklearn import metrics # 导入模型度量
import xlrd  # pip install xlrd==1.2.0
import  pandas  as pd
import numpy as np
```

In [6]:
```
excel_file = '3a.xlsx'
# 读取excel文件里的数据
data = pd.read_excel(excel_file)
```

In [7]:
```
print(data.values)
```

```
[[ 1.     0.697  0.46   1.   ]
 [ 2.     0.774  0.376  1.   ]
 [ 3.     0.634  0.264  1.   ]
 [ 4.     0.608  0.318  1.   ]
 [ 5.     0.556  0.215  1.   ]
 [ 6.     0.403  0.237  1.   ]
 [ 7.     0.481  0.149  1.   ]
 [ 8.     0.437  0.211  1.   ]
 [ 9.     0.666  0.091  0.   ]
 [10.     0.243  0.0267 0.   ]
 [11.     0.245  0.057  0.   ]
 [12.     0.343  0.099  0.   ]
 [13.     0.639  0.161  0.   ]
 [14.     0.657  0.198  0.   ]
 [15.     0.36   0.37   0.   ]
 [16.     0.593  0.042  0.   ]
 [17.     0.719  0.103  0.   ]]
```

In [16]:
```
# y是好瓜、坏瓜
# x是密度、含糖量两个特征
x = data[data.columns[1:3]]
y = data['好瓜'].values
```

In [17]:
```
print(x.shape)
print(y.shape)
```

```
(17, 2)
(17,)
```

# 实验步骤

1. 导入依赖包

2. 导入数据集

3. 划分训练集和验证集

4. 使用逻辑回归对训练集进行训练

5. 对验证集进行预测

In [38]:
```
x_train, x_test, y_train, y_test = model_selection.train_test_split(x, y, test_size = 0.1)
```

In [39]:
```
# 划分训练集和测试集
print(x_train.shape)
print(y_train.shape)
print(x_test.shape)
print(y_test.shape)
```

```
(15, 2)
(15,)
(2, 2)
(2,)
```

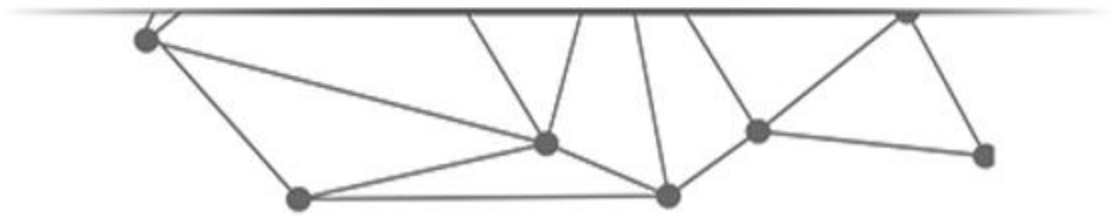
In [35]:
```
# 训练
Lg_R= LogisticRegression()
Lg_R.fit(x_train, y_train)
```

In [28]:
```
y_predict=Lg_R.predict(x_test)
print(y_predict)
print(y_test)
```

```
[0 0]
[0 1]
```



# 拓展—梯度下降法实现



# 损失函数

它是一个函数，用于衡量模型对任何给定数据的性能。损失函数将预测值与期望值之间的误差进行量化，并以单个实数的形式表示出来。

Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

Parameters: $\theta_0, \theta_1$

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

# 梯度下降



梯度下降法是一种求解函数局部极小值的迭代优化算法。

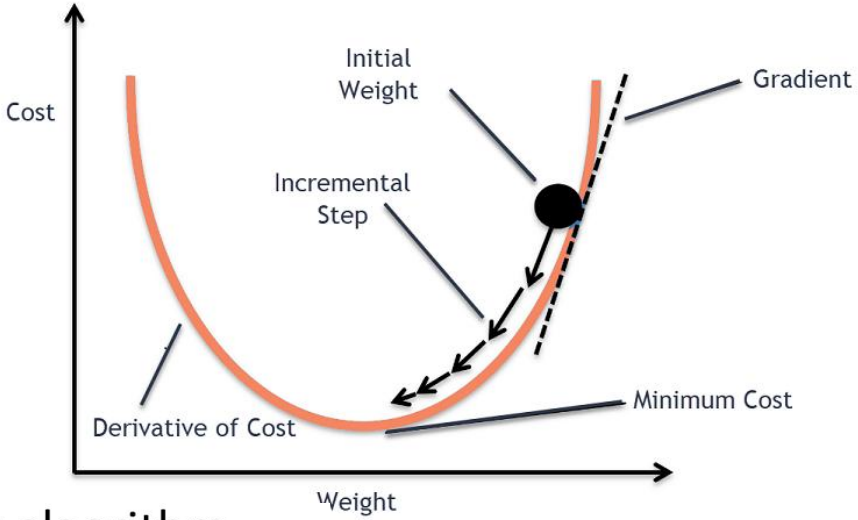
Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

Parameters: $\theta_0, \theta_1$

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

梯度下降算法的目标是最小化给定函数（比如损失函数）。为了实现这一目标，它迭代地执行两个步骤：




1. 计算梯度（斜率），函数在该点的一阶导数；

2. 在与梯度相反的方向上移动一步（移动）

Gradient descent algorithm

$$\text{repeat until convergence } \{$$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

$$(\text{for } j = 1 \text{ and } j = 0)$$

$$\}$$

# 梯度下降

必须谨慎选择，以达到局部最小值。

- 如果学习率太高，我们可能会超过最小值，而不会达到最小值
- 如果学习率太低，训练时间可能会太长
- 学习率最优，模型收敛到最小
- 学习速度太小，需要更多的时间，但会收敛到最小值

代码实现：

参考链接1

参考链接2

